One bit stops the walk: cycle detection in a two-header tape machine

HaoWei Luo*

November 2026

Abstract

A tape machine with two headers, a window and three trims answers a query at each step; in the companion paper its cost is measured on the tapes whose walk *meets*. This paper asks how many of them do, and what the least possible memory is that turns the rest into runs with an answer. Over 18,000 measured (rule, n, tape) cells, 14.9% of the runs meet and 61.9% close a period and then repeat it for ever, dying at the step cap; the cycle is short, a median of two steps. One bit — the answer to "has this window occurred before?" — takes the meeting rate to 37.8% and rescues 36.9% of the runs that otherwise die. The bit is not state: the visited set is the oracle's, the machine keeps $O(\log n)$ bits. It buys termination and no cheapness: a run that used to die now continues to an answer, trims cascade (a median rescued tape needs a few hundred, the worst seen 1,956), and the read bound is $O(n^3)$, no better than the most expensive rule in the catalogue. We also record why one bit of *ink* — a writable mark per cell — buys nothing at all: a mark read costs what a cell read costs, so a mark can only pay by answering a question about several cells at once, and one bit cannot.

1 The problem

The companion paper [1] studies a tape machine whose walk is a pair of headers enclosing a window on a tape of n three-valued cells. At each step a fixed rule, given the window's content and the geometry, names one of eight actions: move a header inward or outward, trim a cell from the left, the right or both, or halt. A tape *meets* when the two headers coincide; only a meeting tape carries an answer δ, and every cost in that paper is conditional on meeting.

The condition is not a formality. Measuring the walk of twenty rules on the standard ensemble (300 tapes per cell, $n = 512, 1024, 2048$; Section 4) gives 18,000 tape-runs in sixty $(rule, n)$ cells, of which 2,681 (14.9%) meet and 11,146 (61.9%) close a period and then repeat it for ever, hitting the step cap without an answer. For most rules the meeting rate at $n = 512$ lies between 2.3% and 31.7% (Table 1); only `monotone` and `hashmod` meet on every tape. The companion paper's means are therefore taken over a small and rule-dependent subpopulation, and its tables do not report the share (its appendix gives pal's meeting rate, 43 of 300 draws at $n = 512$, but no other rule's).

This paper asks the corresponding question of memory. A walk that repeats a state repeats it for ever, because the next state is a function of the state alone; the walk cannot tell that it has been here before, and no amount of time helps. What is the least memory that lets it tell? We give the machine exactly one bit per state — an oracle answering *has this window occurred before in this run?* — and measure what that buys. The answer has two halves, and the second is negative: it buys termination, and it buys no cheapness.

*ORCID: 0009-0007-7206-7095; `luohaowei@bjtu.edu.cn`.

2 The model

The walk. The companion paper's model, unchanged. A region $[L, R]$, a window $[\ell, r] \subseteq [L, R]$, the state $\sigma = (\ell, r, L, R)$; eight actions, three of which trim a cell and reset both headers; a tape meets when $\ell = r$. The rule is a function $f(\varphi(W), n, \ell, r, L, R)$ with φ a fixed function of the window's content.

The bit. Add one query, available at every state:

$$\text{seen}(\ell, r, L, R) = 1 \iff \text{this quadruple has occurred earlier in the same run.}$$

A rule may branch on it; on a repeat it may trim and continue. Nothing else is added. The visited set is the oracle's, not the machine's: the machine's own state remains $O(\log n)$ bits, and it cannot store, count or name the states it has seen. In particular the oracle does not answer *how many times* a window has occurred, and the rule cannot recover a count from it (Section 5).

The trim side. The bit says that the state has occurred; it does not say which end to trim. This is a real degree of freedom: trimming the left end and trimming the right differ by a median of 0.010 in meeting fraction and by at most 0.107 (`AMAX`, `AMIN`). The model therefore declares the side rather than leaving it to the implementation: on a repeat the rule trims the side its own table selects for that state — the same parity rule it uses when it cuts — one cell per repeat. The repair is a trim in the companion paper's sense: it removes the boundary cell of the *tape*, so the region shrinks. It is not a withdrawal of the window, which the walk could undo by widening it again; the difference matters in Section 3.

Lemma 1 (Containment)**.** *Every rule of the companion paper is a rule of this model with the same cost on every tape. A rule that halts without an answer when the bit is* 1 *behaves exactly as in the companion paper: the tapes on which it halts without an answer are the tapes the companion paper does not count either.*

Proof. A rule that never reads the bit is a companion-paper rule. For the second sentence, the run of such a rule is the companion-paper run up to the first repeat; at the repeat it halts, and the tape is one on which no answer is produced, which is precisely a tape the companion paper excludes. The same holds for a rule that trims on the bit rather than halting: a repeated quadruple means the walk is periodic from there on, so such a tape never meets in the companion paper either, and the bit is silent on every tape both models count. □

So the bit cannot change the cost of a tape both models count (Lemma 1). It can only change the *domain*: the set of tapes on which an answer is produced. That is the whole of its effect, and Section 4 measures it.

3 Termination, and what it costs

Theorem 2. *If a rule trims a tape cell whenever the bit is* 1*, every run halts in at most* $n+n(n^2+1)$ *steps.*

Proof. Every trim removes a cell of the tape — the rule's own cuts and the repair alike — so there are at most n trims in the whole run. Between two of them the region $[L, R]$ is fixed, so the state is the pair (ℓ, r) with $L \le \ell \le r \le R$: at most n^2 states. If the walk returned to a state it had

already visited, the bit would be 1 there and the rule would trim, so no state repeats between trims; hence at most n^2 steps between trims, and with at most n trims the run has at most $n + n(n^2 + 1)$ steps. □

Corollary 3. *For the collapse family, where a step costs one read and a trim one more, the read cost of a terminating run is $O(n^3)$.*

The bound of Theorem 2, and of Corollary 3 for the reads, is not an improvement: it is no better than the most expensive rule in the published catalogue, whose cheapest family costs $n + S$ and whose most expensive costs n^3. The bit guarantees that a run ends, not that it ends cheaply, and two features make that concrete. First, the repair itself costs: a median rescued tape needs a few hundred trims and the worst needs 1,956 (`NBLK=1`, $n = 2048$), each paid at one read plus the head re-read. Second, the rescued runs are the expensive ones: a tape that used to die at the cap now continues to an answer and pays for the continuation, so the median reads over all tapes rise even as the meeting rate does.

4 The measurement

Harness. The ensemble of the companion paper: 300 random tapes per cell, $n = 512, 1024, 2048$, one charge for every rule so that the two models are comparable (one read per cell read, counted with multiplicity; the oracle bit is free; a trim pays the cell it gives up plus the head re-read). A cycle is a repeated quadruple; since the tape is fixed, the quadruple determines the next state, so a repeat is a closed period and the run then dies at the cap ($40n^2 + 100$ steps). The repair is the trim of Section 3: on a repeat the rule removes the boundary cell of the tape, on the side its own table selects, and continues.

The gates. Before any number was recorded: 99/99 cells exact against the witness panel's cells at $n = 512$ (reads, δ, end); 114/114 (δ, end) against the reference engines on six random tapes and twenty rules; 800/800 detected periods re-run for a full period; 30/30 tape-runs matching a second, independently written engine on all six fields of the repair; and, on every one of the 18,000 tape-runs, the no-memory walk reproduced field by field against the earlier run (0 mismatches on seven fields), which is what makes the two runs comparable. The repair never fires more than n times (Table 2), no run hit the step cap, and no cell ran out of oracle table.

What the bit buys. Over the 18,000 tape-runs the meeting rate goes from 14.9% to 37.8%, and 4,115 of the 11,146 cap-diers are rescued (36.9%). No run under the bit hit the cap. The rescue is uneven: it is 1.000 for `hashmod` and `monotone`, which never cycle, and between 0.28 and 0.50 for the rules that do, the largest for `dense2` (0.503 at $n = 512$) and the smallest for `runs` (0.276).

The shape of a cycle. The walk does not wander before it repeats: the median period is **two steps**, a move out and the same move back, at every size and for most rules. The width at which the repeat happens is rule-specific (Figure 1) — five cells for `pal`, eleven for `longrun`, about a third of the tape for `NBLK>2` and `BAL>m/2` (749 and 720 cells at $n = 2048$). A long period is the exception: `dense2` cycles in 13–14 steps, `pattern` in six.

The two rules that never need it. `monotone` has no trim and increases ℓ every step, so no quadruple can repeat; it meets all 300 tapes in $n - 1$ steps at $2n - 1$ reads, and the bit is worth exactly nothing. `hashmod` meets all 300 tapes in both models, so again nothing.

	BTHM-0		cycle		BTHM-C (cut)			reads	
rule	meet	cap	len	width	meet	rescued	trims	0	C
`canonical`	0.170	0.620	2	98	0.393	0.360	36	23483	36440
`runs`	0.237	0.447	2	16	0.360	0.276	16	21610	48970
`dense2`	0.187	0.503	14	104	0.440	0.503	19	12540	54711
`peak`	0.070	0.780	2	132	0.340	0.346	73	3312	47816
`trip`	0.080	0.643	2	102	0.297	0.337	112	4730	29460
`oddrun`	0.090	0.697	2	121	0.303	0.306	66	4239	35812
`pattern`	0.317	0.283	6	17	0.420	0.365	4	25280	43399
`LAS`	0.077	0.733	2	108	0.447	0.504	91	7555	41568
`MAXP`	0.117	0.470	4	106	0.297	0.383	108	7572	27860
`KCHG`	0.057	0.777	2	138	0.333	0.356	116	3143	42217
`BAL>m/2`	0.057	0.840	2	190	0.393	0.401	113	3163	36552
`BAL>m/4`	0.040	0.850	4	189	0.357	0.372	124	4038	40818
`NBLK=1`	0.033	0.773	4	168	0.263	0.297	151	3103	27590
`NBLK>2`	0.023	0.857	4	205	0.343	0.373	141	2886	35440
`NBLK>3`	0.023	0.847	4	218	0.350	0.386	118	2636	37686
`AMAX`	0.037	0.703	4	128	0.293	0.365	45	2726	23096
`AMIN`	0.063	0.697	4	138	0.287	0.321	54	5484	24324
`longrun`	0.220	0.063	2	11	0.267	0.737	6	21604	20221
`pal`	0.143	0.260	2	5	0.270	0.487	29	32303	22426
`hashmod`	1.000	0.000	–	–	1.000	–	–	65514	65514
`monotone`	1.000	0.000	–	–	1.000	–	–	1023	1023

Table 1: One row per rule at $n = 512$, 300 tapes, with the cut repair. *BTHM-0*: the share of tapes that meet, and the share that close a period and die at the cap. *cycle*: the median period length in steps and the median window width at the repeat. *BTHM-C*: the meeting share, the rescued share of the cap-diers, and the median number of repair trims a rescued tape needs. *reads*: the median over the tapes the companion paper counts, and over all tapes under the bit.

4.1 The repair, and what it decides

The repair is where the model is tested, in three ways.

It decides whether there is a theorem. The termination proof of Section 3 needs every repair to consume a cell of the tape. A cheaper repair suggests itself: retract the window one cell and leave the tape untouched. Measured, that repair is far better at rescuing runs — 74.0% of them meet against 37.8% — and it hits the cap on no tape either; but the walk can undo it by re-widening the window, so its repair count has no bound (3,627,840 trims at $n = 2048$, against 1,956 for the cut) and no termination theorem applies to it. This paper reports the cut, because a model whose termination is observed but not proved is a weaker claim than one whose termination is proved.

It decides the rescue rate, by a factor of two. Table 3 gives the two repairs side by side at $n = 512$: the retraction rescues up to three times as many runs on the rules that cycle most (`peak` 0.813 against 0.340, `BAL>m/2` 0.887 against 0.393), and the gap is a property of the repair, not of the bit.

Not every repair can work. A repair that only reverses the last move does not terminate: it returns to a state that has also been visited, so the bit fires again and the rule reverses again, and the walk alternates between two seen states. Escaping needs a change in the geometry, and a reversal changes none.

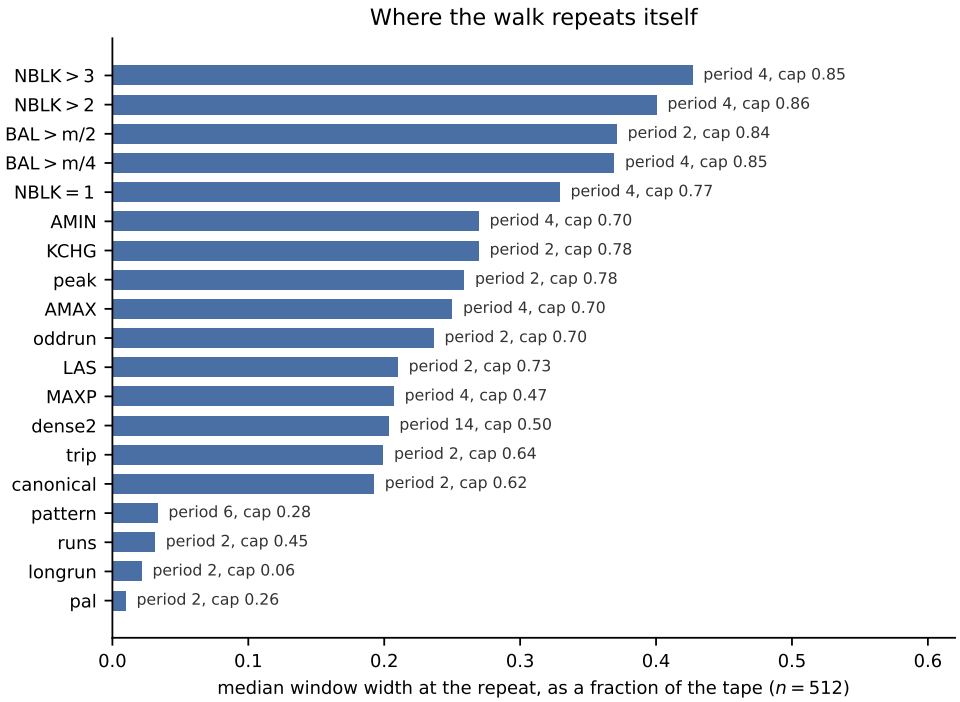


Figure 1: Where the walk repeats itself, at $n = 512$: the median window width at the repeat as a fraction of the tape, with the median period written beside each bar. Four rules repeat only after the window has reached a third of the tape or more, and those are the rules whose repairs are most expensive.

4.2 What the bit costs, and why it is not state

Three accountings could hide in the bit, and the paper fixes all three.

It is not stored state. The set of visited quadruples has size $\Theta(n^4)$, so no machine with $O(\log n)$ bits can hold it: the oracle answers about a set the machine could never carry. The model is therefore not the companion paper with a larger state budget in disguise.

It costs no reads. The query is free. What the bit changes is the number and the length of runs, and the price is paid in trims: each costs one read for the cell it gives up plus the head re-read, and the trim counts are in Table 1.

It does not count. The oracle answers a Boolean, so a rule cannot recover a visit count, its parity, or a colour from it. The counting variant is a different model, and Section 5 says why.

5 Why an oracle, and not ink

The obvious alternative is to give the machine a writable mark per cell instead of an oracle: one bit beside each cell's data, readable with it, writable at the cost of one write. It buys nothing, and the reason is structural rather than a matter of the rules tried.

A read returns the cell's data and its mark together for one read, so consulting a mark costs exactly what consulting the cell costs. A mark can therefore pay only by answering a question about *several* cells at once. One bit can do that in two ways. It can encode a fixed-offset relation, which pays only where the charge is already per pair — and `pal` is already that case, one comparison

n	BTHM-0 meet	cap	BTHM-C meet	rescued	max trims
512	0.152	0.592	0.373	0.373	485
1024	0.148	0.621	0.375	0.365	949
2048	0.146	0.644	0.385	0.370	1,956

Table 2: The aggregate at the three sizes, over the twenty rules that were run (`monotone` is analytic and enters no cell). The meeting rate falls slowly with n while the cap share rises: divergence is not a small-size artefact. The last column is the largest number of repair trims a single run needed; it is always at most n, as the theorem requires.

rule	retraction	cut	Δ
`canonical`	0.690	0.393	-0.297
`runs`	0.670	0.360	-0.310
`dense2`	0.573	0.440	-0.133
`peak`	0.813	0.340	-0.473
`trip`	0.703	0.297	-0.407
`oddrun`	0.743	0.303	-0.440
`pattern`	0.600	0.420	-0.180
`LAS`	0.777	0.447	-0.330
`MAXP`	0.543	0.297	-0.247
`KCHG`	0.827	0.333	-0.493
`BAL>m/2`	0.887	0.393	-0.493
`BAL>m/4`	0.880	0.357	-0.523
`NBLK=1`	0.770	0.263	-0.507
`NBLK>2`	0.857	0.343	-0.513
`NBLK>3`	0.847	0.350	-0.497
`AMAX`	0.740	0.293	-0.447
`AMIN`	0.760	0.287	-0.473
`longrun`	0.277	0.267	-0.010
`pal`	0.400	0.270	-0.130
`hashmod`	1.000	1.000	+0.000

Table 3: The repair decides the rescue rate: the window retraction against the cut, at $n = 512$, as the share of tapes that meet. The retraction is better everywhere and far better on the rules that cycle most, and it is the repair for which no termination bound holds.

being one read for two cells. Or it can carry a monotone Boolean range summary, which pays only where the predicate is not already free — and `pattern`'s predicate is free in its engine. Measured on the two rules for which the record has real machines: both exact variants of `pal` read exactly the baseline at every size (ratio 1.0000) and only spend ink; its one worthwhile certificate, the diagonal, needs about 27 bits at $n = 512$ and recurs a median of 129 steps later, so an untagged bit is stale exactly when it would be useful; a position-only version of the same idea reads 12.6–21.6 times more and returns the wrong answer on 91/85/98 of 300 tapes. `pattern` has one working mechanism, an adjacency bit on the paired opening pass, worth 190/383/768 reads — 0.25% of the charge falling to 0.07% — fitted exponent 1.009 against a charge that fits 1.963: it does not scale. Even an unbounded, perfectly tagged certificate store would save 17.1/18.0/18.6% of the charge, and the plane holds about 18 of the roughly 5,740 certificates one tape needs.

A larger plane is not a larger version of this model; it is a different one, and its question is different: what does $O(n)$ bits of state, on the tape, buy over $O(\log n)$? That question is not

answered here.

6 What this says about the companion tables

Two consequences, and the second is a correction of interpretation rather than of number.

First, the companion paper's costs are not the costs of the model on all tapes; they are the costs on the 14.9% that meet, and the rate varies by a factor of forty across rules (from 2.3% for the level-count rules to 100% for `hashmod`). Any comparison between two rules' means is therefore a comparison of two different subpopulations, and a revision should print the rate beside the mean.

Second, divergence is not a defect of particular rules: it is what the model does. The walk is memoryless, so a rule that can return to a state will return to it, and the state space is $O(n^4)$. Of the twenty rules measured, eighteen cycle on some tapes, and for fifteen of them the cycle is the majority outcome at some size. What saves the meeting tapes is geometry, not design: the walks that shrink monotonically cannot repeat, and the walks that end quickly escape the cap.

7 Open

Can a machine hold the bit? The oracle answers about a set of size n^4; no machine of the class can store it. For a monotone walk the window is determined by the step count, so the bit is free and the walk never needs it. For a general walk the question is whether some rules can recognise their own repeats with $O(\log n)$ bits — a question about the rule, not about the model, and one this paper does not answer.

Counting instead of remembering. An oracle that answers *how many times* (or the count modulo k) is a different model: its memory is $O(n^2 \log k)$ bits, it is Langton's-ant-shaped [2], and the interesting question there is not cost but computational power. It should not share a paper with this one.

The domain, as a quantity. The companion paper reports a cost; this paper reports a domain. A model that reports both — the cost on the tapes it answers, and the share of tapes it answers — would be strictly more informative than either, and the tables here are a first instance of the second column.

A Engines and gates

The walk is implemented once, as a table-driven machine (the action table and the two predicates b_1, b_2 of each rule, taken from the reference engines), and charged with one accounting for every rule so that the two models are comparable; the reference engines charge differently (band, `palmach2`, `hk_engine`), and mixing them would confound the model with the engine. `hashmod` is the one rule whose reference machine is not the common engine; it is nevertheless the same walk, and its (δ, end) agrees with the table-driven engine on all six check tapes and its meeting count on all 300 tapes at every size. `runs`, `pal`, `longrun` and the band rows were checked the same way.

References

[1] H. Luo. A memoryless two-header tape machine: what a window costs, and why two lower-bound techniques fail. Preprint, version 2.0, doi:10.5281/zenodo.24199236, 2026.

[2] C. G. Langton. Studying artificial life with cellular automata. *Physica D* 22 (1986), 120–149.